

EcoPulse Deployment Readiness

Smart Energy Grid & Carbon Credit Marketplace

Field	Value
Document version	1.0
Date	May 2026
Current maturity	Demonstration / local-dev MVP
Production status	Not production-ready

Executive Summary

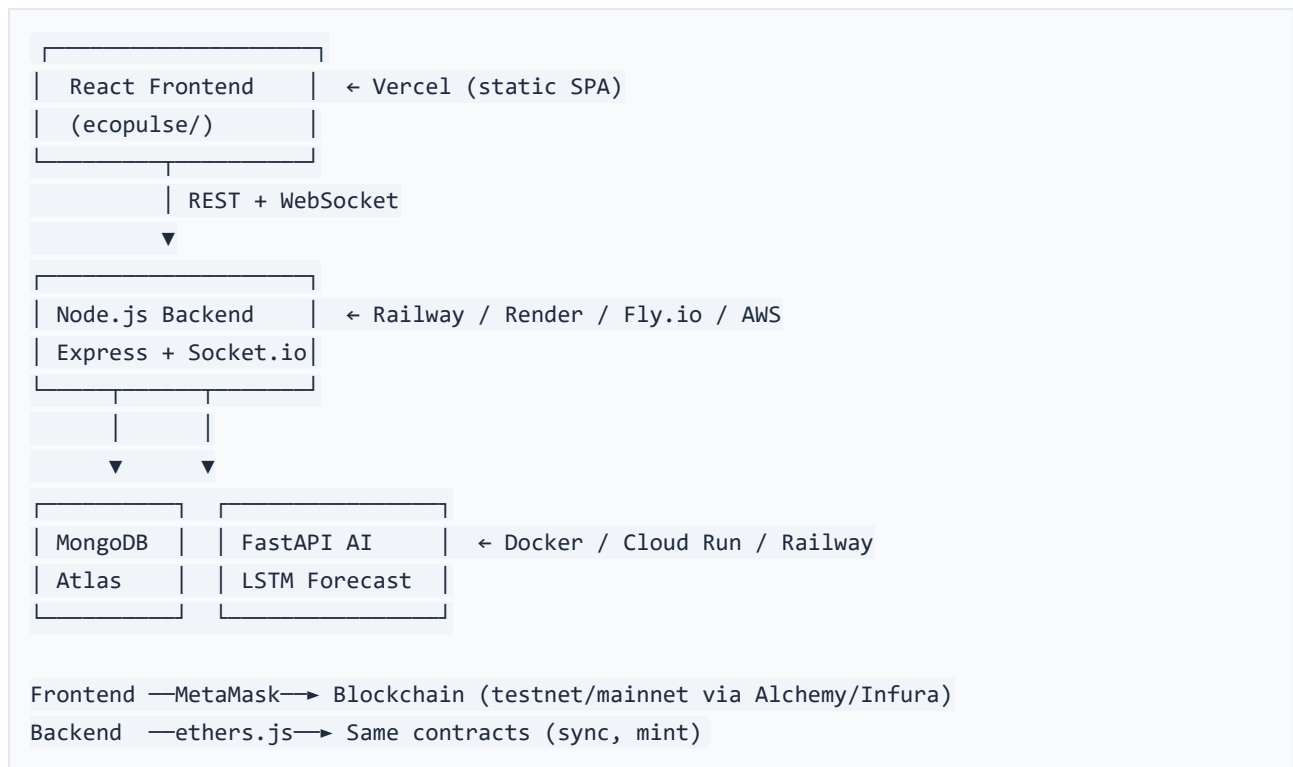
EcoPulse is a full-stack decentralized energy platform combining a **React (Vite) frontend**, **Node.js/Express backend** with **Socket.io**, **MongoDB**, a **FastAPI AI forecasting service** (LSTM), and **Solidity smart contracts** for peer-to-peer energy trading and carbon credits.

Key finding: The project is suitable for demos, coursework, and local development. Most data flows use **simulated or dummy inputs**. Security, hosting, and blockchain layers require substantial work before a public deployment on **Vercel** or any production domain.

Hosting model:

- **Vercel** can host **only** the static frontend (`ecopulse/`).
 - The **backend**, **AI service**, **MongoDB**, and **blockchain RPC** must run on separate infrastructure.
-

1. Architecture Overview



Repository layout:

Directory	Role
ecopulse/	Vite app entry, routing
frontend/	Shared pages, components, utils
backend/	Express API, Socket.io, Mongoose
ai_service/	FastAPI LSTM forecasting
contracts/	CarbonCredit.sol, EnergyTrading.sol
docs/	Architecture and this document

2. What Is Done Today (Implemented)

The following features are **implemented and functional** in local development.

2.1 Authentication

Feature	Status	Location
User registration	Done	backend/controllers/authController.js
Login with JWT	Done	backend/routes/auth.js
Password hashing (bcrypt)	Done	backend/models/User.js
Protected /auth/me	Done	backend/middleware/auth.js
Frontend auth context	Done	frontend/context/AuthContext.jsx
Login/Register pages	Done	frontend/pages/Login.jsx , Register.jsx
Route guards (UI only)	Done	frontend/components/ProtectedRoute.jsx

2.2 Energy Grid Data

Feature	Status	Location
Energy node CRUD	Done	backend/models/EnergyNode.js , nodeController.js
Energy readings (POST/GET)	Done	backend/models/EnergyReading.js , readingController.js
Real-time Socket.io broadcast	Done	backend/server.js
Live dashboard chart	Done	frontend/pages/Dashboard.jsx , EnergyChart.jsx
Energy simulator (DB-backed)	Done	backend/simulator.js

2.3 Analytics (Recent)

Feature	Status	Location
Summary analytics API	Done	GET /api/v1/analytics/summary
Energy totals aggregation	Done	backend/services/analyticsService.js
Active nodes stats	Done	GET /api/v1/analytics/nodes
Trade activity stats	Done	GET /api/v1/analytics/trades
Carbon credit stats	Done	GET /api/v1/analytics/carbon
Platform status endpoint	Done	GET /api/v1/analytics/status
Blockchain event sync	Done	backend/services/blockchainSyncService.js
Dashboard wired to APIs	Done	frontend/pages/Dashboard.jsx
Credits page with sync	Done	frontend/pages/Credits.jsx

2.4 AI Forecasting

Feature	Status	Location
FastAPI forecast service	Done	<code>ai_service/main.py</code> , <code>routes/forecast.py</code>
LSTM model training script	Done	<code>ai_service/train.py</code>
Backend forecast proxy	Done	<code>backend/controllers/forecastController.js</code>
Forecasts UI page	Done	<code>frontend/pages/Forecasts.jsx</code>
MongoDB field alignment	Done	<code>ai_service/utils/database.py</code> (energyreadings)

2.5 Blockchain & Trading

Feature	Status	Location
CarbonCredit ERC20 contract	Done	<code>contracts/CarbonCredit.sol</code>
EnergyTrading P2P contract	Done	<code>contracts/EnergyTrading.sol</code>
Hardhat deploy module	Done	<code>ignition/modules/EnergySystem.js</code>
Frontend MetaMask integration	Done	<code>frontend/utils/blockchain.js</code>
Trading page (list/buy/approve)	Done	<code>frontend/pages/Trading.jsx</code>
Backend blockchain service	Done	<code>backend/services/blockchainService.js</code>
Contract unit tests	Done	<code>test/EnergySystem.test.js</code>

2.6 Testing & Tooling

Feature	Status	Location
MVP E2E test script	Done	<code>backend/scripts/test-mvp.js</code>
Health check endpoint	Done	<code>GET /api/health</code>
Global error handler	Done	<code>backend/middleware/errorHandler.js</code>
Request logging	Done	<code>backend/middleware/logger.js</code>

3. What Is Dummy, Test, or Not Production-Ready

These items **must be understood** before treating the app as production-ready.

3.1 AI / Machine Learning

Issue	Detail
Dummy forecast data	AI service generates synthetic sin/cos data when <code>use_dummy_data=true</code>
Auto-fallback	Backend uses dummy when MongoDB has fewer than 30 readings
Model artifacts missing	<code>ai_service/models/saved/</code> (<code>.keras</code> , <code>scaler.save</code>) not in repository
Training uses dummy	<code>ai_service/train.py</code> trains on synthetic data by default
No scheduled retraining	No cron/job to refresh model from live readings
No model versioning	No A/B or rollback strategy

Files: `ai_service/utils/database.py` , `ai_service/routes/forecast.py` ,
`backend/controllers/forecastController.js`

3.2 Energy Data

Issue	Detail
No real IoT ingestion	Documented in <code>docs/architechture.md</code> but not implemented
Socket simulator	<code>simulate_nodes.js</code> emits readings without auth or DB (unless valid ObjectId)
HTTP simulator	<code>simulator.js</code> generates random kW values per node type
No weather/oracle data	Listed in <code>production_time.md</code> , not in code

3.3 Blockchain

Issue	Detail
Local Hardhat only	Chain ID 31337; no testnet/mainnet in <code>hardhat.config.js</code>
Default dev private key	Hardhat account #0 used if <code>PRIVATE_KEY</code> unset (<code>blockchainService.js</code>)
Dev mint button	Trading page exposes <code>mintDevTokens</code> for local testing
No contract audit	Required before mainnet per <code>P2P_Trading_Production_Readiness.md</code>
Full-block scan sync	<code>blockchainSyncService.js</code> queries from block 0 each sync (not scalable)
MongoDB ↔ chain decoupled	Trades indexed separately; no automatic credit mint on generation

3.4 Frontend / UX

Issue	Detail
Settings page stub	<code>frontend/pages/Settings.jsx</code> — placeholder text only
JWT not sent to API	<code>fetchApi</code> supports token but pages do not pass it
Hardcoded localhost URLs	Fallback <code>http://localhost:5000</code> in several files
<code>recharts</code> dependency gap	Used in <code>EnergyChart.jsx</code> but missing from <code>ecopulse/package.json</code>
No <code>vercel.json</code>	SPA routes may 404 on direct navigation

3.5 Security

Issue	Detail
Public write APIs	Nodes, readings, analytics, sync — no <code>protect</code> middleware
Open CORS	<code>cors()</code> with no origin restriction
Socket.io <code>origin: '*'</code>	Any site can connect
AI CORS <code>allow_origins=["*"]</code>	<code>ai_service/main.py</code>
JWT in localStorage	XSS exposure risk
No rate limiting	No <code>helmet</code> , <code>slowapi</code> , or <code>express-rate-limit</code>
Roles unused	User model has roles; routes never enforce
<code>userId</code> from request body	Nodes can be created for any user ID

3.6 Operations

Issue	Detail
No CI/CD	No GitHub Actions or similar
MongoDB optional at startup	Server starts even if <code>MONGO_URI</code> missing
<code>app.log</code> in repo	Runtime log file tracked in <code>ai_service/</code>
No monitoring	No Sentry, Datadog, or OpenTelemetry
Analytics status lies	Reports MongoDB connected unconditionally

4. Pre-Deployment Checklist

Use this checklist before pointing any public domain at the application.

P0 — Blockers (Must complete before any public URL)

- ☐ **Split hosting:** Deploy backend + AI off Vercel (Railway, Render, Fly.io, AWS, GCP Cloud Run)
- ☐ **MongoDB Atlas:** Set `MONGO_URI`; change `backend/config/db.js` to **fail fast** if DB unavailable
- ☐ **Secrets:** Set strong `JWT_SECRET`; never use default Hardhat `PRIVATE_KEY` in production
- ☐ **Protect APIs:** Apply `protect` middleware to `/nodes`, `/readings`, `/analytics`, `POST /sync`
- ☐ **Frontend auth:** Pass JWT from `AuthContext` in all `fetchApi` calls
- ☐ **CORS:** Restrict Express and Socket.io origins to production domain(s)
- ☐ **Remove dev paths:** Disable `simulateReading`, `mintDevTokens`, or gate behind `NODE_ENV=development`
- ☐ **Fix Vercel build:** Add `recharts` to `ecopulse/package.json`; add `ecopulse/vercel.json` SPA rewrites
- ☐ **Environment templates:** Copy `.env.example` files and fill production values (see Section 6)
- ☐ **AI model artifacts:** Train LSTM, upload `models/saved/` to storage, or run init job on deploy
- ☐ **Verify build locally:** `cd ecopulse && npm run build` on clean `npm install`

P1 — High Priority (Production MVP)

- ☐ **Testnet contracts:** Add networks to `hardhat.config.js`; deploy to Sepolia/Polygon testnet
- ☐ **Dedicated RPC:** Alchemy or Infura; set `RPC_URL` and all `VITE_*` contract addresses
- ☐ **Dockerize AI:** Follow `ai_service/production_guide.md` (Gunicorn, not serverless)
- ☐ **Socket.io scaling:** Redis adapter if running multiple backend replicas
- ☐ **Rate limiting:** Auth, forecast proxy, and public POST endpoints
- ☐ **Settings page:** Profile, wallet linking, notification preferences
- ☐ **CI pipeline:** Hardhat tests + `vite build` + `npm run test:mvp` against staging
- ☐ **Log hygiene:** Remove `ai_service/app.log` from git; add rotation
- ☐ **HTTPS everywhere:** Backend, AI, and WebSocket (`wss://`) endpoints

P2 — Production Hardening

- ☐ **Smart contracts:** ReentrancyGuard, partial fills, expiry, cancellation (`production_time.md`)
- ☐ **Event indexing:** The Graph or live listeners instead of polling from block 0
- ☐ **Wallet UX:** Wagmi/RainbowKit; human-readable revert messages
- ☐ **Monitoring:** Sentry or equivalent on frontend, backend, and AI
- ☐ **Structured logging:** JSON logs with request IDs
- ☐ **Role-based admin:** Enforce `admin` role on sync and sensitive routes
- ☐ **Security audit:** Third-party review before mainnet

P3 — Roadmap (Documented, Not in Code)

- ☐ IoT device integration (`docs/architecture.md`)

- ☐ Weather API for improved forecasts
- ☐ Mobile application
- ☐ Federated learning across nodes
- ☐ Chainlink oracle pricing
- ☐ Off-chain order book (per `P2P_Trading_Production_Readiness.md`)

5. Recommended Hosting Map

Service	Platform	Root / Command	Notes
Frontend	Vercel	<code>ecopulse/</code> , <code>npm run build</code>	Static SPA only
Backend	Railway, Render, Fly.io	<code>backend/</code> , <code>npm start</code>	Requires WebSocket support
AI service	Cloud Run, Railway (Docker)	<code>ai_service/Dockerfile</code>	TensorFlow; long cold starts
Database	MongoDB Atlas	—	Shared by backend + AI
Blockchain	Sepolia / Polygon + Alchemy	—	Redeploy contracts; update env vars

Do not deploy to Vercel:

- `backend/server.js` (Express + Socket.io + background sync)
- `ai_service/` (Python + TensorFlow)
- Hardhat local node
- MongoDB (use Atlas)

6. Environment Variables Reference

See `.env.example` files in:

- `backend/.env.example`
- `ecopulse/.env.example`
- `ai_service/.env.example`

Backend (backend/.env)

Variable	Required	Description
MONGO_URI	Yes (prod)	MongoDB Atlas connection string
JWT_SECRET	Yes	Strong random secret for signing tokens
JWT_EXPIRE	No	Token expiry (default 7d)
PORT	No	Server port (default 5000)
NODE_ENV	Yes	production hides error stacks
AI_SERVICE_URL	Yes	e.g. https://ai.yourdomain.com
RPC_URL	If using chain	Alchemy/Infura HTTPS URL
PRIVATE_KEY	If using chain	Server wallet (never commit)
CARBON_CREDIT_ADDRESS	If using chain	Deployed contract address
ENERGY_TRADING_ADDRESS	If using chain	Deployed contract address
BLOCKCHAIN_SYNC_INTERVAL_MS	No	Default 60000

Frontend — Vercel (ecopulse/.env)

Variable	Required	Description
VITE_API_URL	Yes	e.g. https://api.yourdomain.com/api/v1
VITE_SOCKET_URL	Yes	e.g. https://api.yourdomain.com (use wss if supported)
VITE_CARBON_CREDIT_ADDRESS	If using chain	Testnet/mainnet address
VITE_ENERGY_TRADING_ADDRESS	If using chain	Testnet/mainnet address

AI Service (ai_service/.env)

Variable	Required	Description
MONGODB_URI or MONGO_URI	Yes	Same Atlas cluster as backend
PORT	No	Uvicorn port (default 8000)

7. Vercel Deployment Steps

7.1 Project setup

1. Import repository to Vercel.
2. Set **Root Directory** to `ecopulse`.
3. **Build Command:** `npm run build`
4. **Output Directory:** `dist`
5. **Install Command:** `npm install`

7.2 Environment variables (Vercel dashboard)

Add all variables from `ecopulse/.env.example` with production URLs pointing to your deployed backend.

7.3 SPA routing

Create `ecopulse/vercel.json`:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

7.4 Pre-deploy verification

```
cd ecopulse
npm install
npm run build
```

Fix any missing dependencies (e.g. `recharts`) before first deploy.

7.5 Post-deploy checks

- ☐ Login and register work against production API
 - ☐ Dashboard loads analytics (not only socket)
 - ☐ WebSocket connects (`wss` if applicable)
 - ☐ Trading page connects MetaMask to correct network
 - ☐ Forecasts page returns data or graceful 503
-

8. Backend & AI Deployment Notes

Backend

- Must support **long-running process** and **WebSocket** upgrades.
- Set `NODE_ENV=production`.
- Ensure Atlas IP allowlist includes backend host IP (or `0.0.0.0/0` for PaaS).
- Health check: `GET /api/health`.

AI Service

- Use Docker (`ai_service/Dockerfile`) with Gunicorn + Uvicorn workers.
- Mount or copy `models/saved/` at deploy time.
- Set `MONGODB_URI` to same database as backend.
- Health check: `GET /`.

Blockchain (optional for MVP)

1. Start with **testnet** (Sepolia).
2. Deploy via Hardhat Ignition with network config.
3. Copy addresses to backend and Vercel env vars.
4. Fund server wallet with test ETH.
5. Run `POST /api/v1/analytics/sync` after deploy (or wait for background sync).

9. Testing and Sign-Off Criteria

Before go-live, confirm:

Test	Command / Action	Expected
Contract tests	<code>npx hardhat test</code>	All pass
MVP API test	<code>cd backend && npm run test:mvp</code>	All checks pass (AI may warn offline)
Frontend build	<code>cd ecopulse && npm run build</code>	No errors
Manual auth flow	Register → Login → Dashboard	JWT stored; UI protected
Analytics	<code>GET /analytics/summary</code>	Real totals from DB
Security review	Inspect public POST routes	None without auth in prod
Secrets audit	Production env	No Hardhat default keys

10. Estimated Effort (Planning Guide)

Phase	Scope	Rough estimate
P0	Security, hosting split, env, build fix	1–2 weeks
P1	Testnet, Docker AI, CI, Settings	2–4 weeks
P2	Contracts, indexing, monitoring, audit prep	4–8+ weeks
P3	IoT, mobile, advanced ML	Ongoing roadmap

Estimates assume a small team; contract audit and mainnet add additional calendar time.

11. Related Documentation

Document	Path	Purpose
System architecture	<code>docs/architechture.md</code>	High-level design
P2P production guide	<code>P2P_Trading_Production_Readiness.md</code>	Blockchain UX roadmap
Contract roadmap	<code>production_time.md</code>	Solidity enhancements
AI production guide	<code>ai_service/production_guide.md</code>	Gunicorn, caching, rate limits

12. Document Revision History

Version	Date	Changes
1.0	May 2026	Initial deployment readiness assessment

This document reflects the EcoPulse codebase as of the analytics and sync implementation. Re-run assessment after major feature or security changes.